

Project 6 - Elevator

Design Document

Supervisory Application and FSM

Group 1

Ryan Pratt, Nigel Sinclair, Fergus Page

Introduction

This document describes the design criteria for developing the Elevator's Supervisory Application (SA) and Finite State Machine (FSM) logic. This document will describe in detail the conditions for state transitions, exception behaviour, opposing condition handling, high level data-structures, and alternative finite logic depending on the given hardware.

The elevator project consists of 4 primary controllers and one primary level sensor, that communicate through a CAN bus network to schedule and control the system overall. These controllers are:

1. Supervisory Controller (Raspberry Pi)
2. Elevator Controller (Arduino)
3. Floor Controllers (STM32)
4. Car Controller (STM32)

From a high-level perspective the SA serves as a kind of operating system for the entire machine. Managing resources, collecting system data, dispatching orders, and ensuring the smooth operation of the entire system. As such, both the design of the state machine and the application that surrounds it must prioritize reliable operation and deterministic behaviour. That is to say, the SA must ensure that physical operations are decoupled from the communications, and that the system never enters an undefined state.

Equipment Description

The elevator project comprises several subsystems that the FSM must manage to ensure proper coordination across all equipment. Major systems include the supervisor controller, elevator controller, floor controllers, and car controller.

The system architecture places the supervisory controller (SC) as the central control node for the entire network; the other controllers specialize in handling raw sensor data acquisition and transmission. Data collected by supporting controllers is transmitted to the SC via the CAN bus network. Both the car and floor controllers provide user input from button presses and send floor requests to the SC. The elevator controller moves the car up and down to the requested floor and determines the current height of the car via a light distance centre.

The SC runs on a Raspberry Pi 3 (Rpi3), however, for testing purposes the SC was run on a more powerful Ubuntu 64-bit OS was run on a Raspberry Pi 5 (Rpi5). For all design purposes, the specific hardware of the SC doesn't matter, however, specific implementation details in the C++ vary depending on the underlying system. With the modern Rpi5 running C++23, and the older Rpi3 running C++20.

Supervisory Application Design

Introduction

The SA is the main application that handles elevator requests, request scheduling, priority queues, and physical dispatching. Since the SA software lies between the driver code and the front end, we decided to use the C++ programming language. We chose C++ for its ability to support both high-level object-oriented programming and low-level systems programming while keeping high performance. Specifically, we developed the SA using C++23 for its updated language features including formatted printed strings, improved constant expression evaluations, conditional evaluation at compile time, and standard expected features.

The design specification for the SA was based on two fundamental design principles:

1. Communication and hardware operation must be isolated from each other.
2. The specific state of the system must be deterministic in nature.

These design principles are overkill for the specific application of a small project elevator, but stem from the extrapolation of this project to practical real-world considerations. The first design specification is implemented to ensure that a freeze/failure on the CAN bus operation does not interfere or stop the SA from determining the current state of the FSM or running checks on the SA. The second specification follows the industrial nature of this project, such that under all circumstances the elevator must be in a known state. This avoids undefined behaviour and reduces the chances of bugs occurring.

Supervisory Application Software Development

From the start a major concern was the reading and writing of data to the CAN bus while simultaneously controlling the elevator or determining the scheduling. If for example, the CAN bus code got hung up endlessly listening for messages the rest of the application would stop running. No passengers riding the elevator should be stuck because a CAN bus read failed, or vice versa, a floor request shouldn't be ignored simply because the current passenger provided a destination. From this we determined the simplest solution was to separate the two areas of the code. Doing this ensures that even if the two events handle simultaneously neither one is lost because the SA was preoccupied.

The SA separates the two sections of the program using multi-threading and a non-blocking polling loop. Where the control thread polls the comms at fixed intervals during its control loop, while the comms thread continuously monitors the CAN bus. This allows the controls and communications to be isolated from each other, while ensuring that communications aren't missed accidentally. Additionally, by using a simple 250ms heartbeat between threads we can verify the liveliness of the worker thread. This ensures that the SA knows when an internal malfunction has occurred and can restart communications using a new thread to ensure the process is always available.

The multi-threaded approach also helps with the second design specification by ensuring the program is always in a defined state. Since the program can never be hung up on waiting for a CAN message to send or receive. The secondary design specification is intended to ensure the program is always in a defined state. Deterministic behaviour ensures that the program is never undefined, and that all states during the operation of the machine are accounted for. The supervisory application is designed with the intention that future nodes will be setup using a heartbeat protocol to determine comms state and activate an emergency stop/fault procedure.

FSM Logical Operations

This section describes the order, priority, abstract data structures, and exception handling without specifically detailing the full FSM. It's designed to allow programmers to quickly understand the order at which requests are served, how requests are received and stored, and how the FSM should respond to a given input to maintain a defined state at all times.

FSM Design Decisions

Standard Elevator

The standard elevator was designed with the following constraints:

- The floor call button calls the car to a floor (NOT DIRECTIONAL)
- Motor Control Unit is self monitoring; no feedback is provided to upstream nodes.

Following these constraints, the following FSM produces reliable deterministic behaviour.

General Controller Area of Responsibilities

Supervisory Controller

- Network communication
- Front-End hosting
- Floor and car scheduling – first come first serve basis
- Calculate and provide delta distance between floors and dispatch the car to floor

Floor Controllers

- Send car request to supervisory controller for a given floor
- If individual sensors are provided on a per-floor basis floor specific sensor devices should be handled by the designated floor controller

Car Controller

- Send selected destination floor
- Send current floor

Motor Controller

- Receives dispatch commands from supervisory controller, dispatches the car to the corresponding floor
- Takes in emergency stop logic for photo-eye limit sensors

Supervisory Controller States and Conditionals

The supervisory controller (SC) acts as the main coordination and managing hub. It collects and orchestrates data collected via the subsystems and controls how the overall system works. Centralizing the code controlling the system inside the Raspberry Pi (RPi) has notable advantages. Since the Pi also handles front end and database management, all command and control can be run on a single device with minimal downtime caused by sending and receiving messages.

While this document does not describe integration of the database applications in this project, future revisions could be added as required.

The SC by default will follow requests in the following order:

1. Physical button requests from the local lan
2. Network button requests via a remote connection

This follows the principle that in-person requests have a higher priority than remote commands. Floor requests sent (whether physical or via remote connections) are handled using a queue system. Where the first request is handled completely before the next request can be executed.

From a high level the elevator should work in the following machine state:

Current State	Condition	Result State	Action
Idle	Floor request is received or pulled from queue	Collection	A floor request has been received but a car command has not been made.
Dispatching	1. Car command is received 2. No car command is received within 10s of arrival at	1: Arrival 2: Idle	1: The car is still considered to be in use by users, new floor requests are added to queue but are not executed until after the arrival state

	destination		is complete. 2: The car is assumed to have been mis-called/the user opted for the stairs and will pull the next job from the queue. Returns to idle and repeat FSM.
Arrival	Car reaches car command floor	Idle	The car reaches the user's requested floor, and the cargo is considered "delivered". Upon completion the system returns to the idle state.

State Descriptions

IDLE

The idle state is the default state of the elevator, during this state the elevator will actively look for new jobs to execute. Jobs will be pulled from two queues, one for dealing with physical requests from the CAN network, and the other for dealing with remote requests from the RDP. Priority given to the physical request queue over remote requests. This is done with the understanding that people physically at the elevator should take priority over a convenience feature such as remote floor requests

Dispatching

The dispatching state defines the state wherein the elevator has taken a job from the queue (whether physical or remote) and is moving to the requested floor to pick up the requesting user.

If the elevator arrives at the requested floor and does not receive a car command within 10 seconds (a user requested destination floor from the car controller), the elevator returns to the idle state and will look to grab the next job from the available queues.

Arrival

Arrival is the state of the elevator after the requester enters the elevator provides a car command and has arrived at their requested floor. After allowing the passengers onboard to disembark the elevator returns to the Idle state.

Supervisory Application Code Layout

Coding Standards

This project uses C++20 and a custom coding standard combining features from popular standards such as Microsoft and Google. The primary goal of these standards is to ensure that the project is maintainable, easily navigable, and modular for future expansion. While this document will not detail the full agreed upon coding standard, it will briefly cover the core principles the project follows. Namely: Modularity, Descriptive Naming, Limited Macros, Control Flow, and No Inheritance.

Modularity

This program was designed to be a modular program with the rule of thumb being “One function one file”, this design principle essentially demands that if a file exists (such as `can_protocol.cpp`) then all corresponding code associate with that file must belong to that file. This eliminates chasing declarations across the codebase and allows for simpler debugging and code changes. This prevents so called “god-files” that cover too broad of a scope to reasonably managed. In the event that a file exceeds reasonable readability guidelines (~500 lines) the simplest file becomes an interface with more specific sub-implementation files branching off. For example, `comms.cpp` may become the front-facing API with specific implementations being defined in `can.cpp/uart.cpp`.

Descriptive Naming

This program purposefully uses verbose naming schemes for more clearly defined data structures and methods. While this naming scheme is not a strict requirement, it does help new programmers working in the codebase get a feel for the code faster.

Names of classes or data structures use prefixing to more easily distinguish between classes and data structures. This program uses a `c<NAME>` scheme for classes, `ec<NAME>` for enum classes, `s<NAME>` for structures, and `k<NAME>` for constants. Private class variables use a `<NAME>_` scheme to more efficiently distinguish them between regular variables.

Limited Macros

This guideline is more applicable to the C++ application than specific driver code written in C. However, due to the lack of scope or type awareness on C-style macros, when possible, type aware `constexpr` statements should be preferred. This limits the chances of a name collision and the inherit difficulty of debugging such a failure.

Control Flow

The control flow of this program tries to emphasize simplicity over cleverness. Code should be read like instructions with simple control logic where performance is not strictly critical. This reduces debugging times, allows for easier code management, and overall makes the repository easier to work in.

No Inheritance

From the outset this project ensures that there is no inheritance in the codebase. This is largely due to the complications that arise when managing subclasses. It makes the codebase more brittle and reduces the modularity of the program. As such, inheritance is purposely avoided even when its use could be reasonably justified.

Application Design

The program consists of 3 major parts: Entry, Application, and Comms. The main entry point of the program (main.cpp), is responsible for bootstrapping the application environment. Main loads the Socket and configures the comms before the application boots. After booting, the main launches the application which begins the runtime.

Bootstrapper (main.cpp)

Depending on the number of arguments passed to the main function, the bootstrap will try one of the following on start up.

1. No arguments are passed to the application – The bootstrapper will attempt to initialize the CAN socket according to a predetermined default configuration (can0 @ 125kbs). If not run with sufficient privileges, this operation can fail and the bootstrapper will print a std error “supervisory_controller: COMMS initialization failed: <operation error message>” and return 1. Upon a successful initialization and startup the program logs to the console the specific interface started and the bitrate used.
2. Arguments are passed to the application – The bootstrapper will use the provided argument list as the port and requested baud rate (can1 @ 250kbs), this will replace the default configuration, and the program will attempt to open the passed port and baud rate. If using a pre-configured port setup via BASH, the application provides a Boolean value to toggle which will prevent the bootstrapper from attempting to initialize the port again.

After the bootstrapper successfully initializes the comms thread the controls thread is created. The separation of duties between two threads ensures that a failure in the comms does not hold up the main control loop. The main control loop initializes a controller class and runs at a fixed polling execution time of 10ms per cycle. The communication thread is not configured to run a set polling loop, instead it continuously transmits and drains the initialized interface to a ring buffer consumed by the control thread.

This allows the control thread to execute at a deterministic pace, while ensuring the application isn't missing floor requests sent during the polling loops. All requests are recorded to the ring buffer and are executed by the control thread at the start of each loop.

Communication Service (can_comms_service.cpp/.hpp)

The communication service is responsible for moving data between the can adapter (transmit/receiving functionality) and the main control thread. The header file declares a

common interface used to move data and information between the two threads. From a high-level overview, the header creates the comms state/fault reason enumerators and defines the structs CanCommsHealthSnapshot and CanExchange. These are responsible for tracking the current status of the can comms thread, and the exchange format used between the two threads respectively.

The health snapshot tracks the state of the thread, whether the thread is faulted, the current heartbeat status, and the Rx/Tx failures. The can exchange struct creates the two single-producer single-consumer ring buffers used to move data between the controls and comms threads. By design the received events ring buffer is larger (storing up to 64 supervisory events), than the transmit buffer (storing up to 16 can frames to be transmitted). This saves space in the binary with the reasonable assumption that the SC needs to handle more incoming data than generating outgoing control commands.

The heartbeat is implemented via a simple atomic counter via a fetch_add, and load. The communication thread endlessly increments a counter via a fetch add atomic operation, while the supervisory application verifies that the previous counter value is not the same as the current value. If they are, a 250ms timeout will begin. If comms don't resume within the 250ms window the control thread will create a new comms thread to ensure that communications remain available.

Cross Node Safety Features - Heartbeat

Supervisory Application – Heartbeat Node Safety

The supervisory application maintains a heartbeat with the floor controller nodes by periodically polling the CAN bus with heartbeat messages using a custom extended CAN protocol. This is done to ensure that the CAN bus is always active and any failures in the communication system are immediately detected.

The supervisory controller will indiscriminately heartbeat the network at a rate of one beat per 3 seconds. This sends out an extended CAN frame to all nodes on the network, all other nodes on the network will respond to this heartbeat by sending a return message. Incoming extended frames matching a heartbeat signature are handled via a separate queue than the core state-machine control queues.

At the end of each 10ms control loop, the SC checks the heartbeat queues for reply to messages to the CAN heartbeat. Looking for messages from all floor controllers, and the car controller (the elevator controller is excluded due to scope constraints). If the SC has received a message from all other controllers the SC can confirm the CAN bus is working correctly.

The non-SC controllers may message the SC on condition of missing a heartbeat message. These occur at a 5 second timeout of no heartbeat messages received. Upon the timeout elapsing, the specific controller will heartbeat message the SC, which will reply with a

heartbeat ok message to test that the specific node can receive CAN messages. If a corresponding “heartbeat ok” reply is not received within a 1 second verification window the SC will report a communication failure with that specific node.

SC Heartbeat Design

The SC heartbeat is built around a few small data structures in the CAN and application layers. The protocol layer defines a heartbeat type enum, `ecNodeHb`, for messages such as `Ok`, `SupervisorRequest`, `NodeRequest`, and `Error`. Heartbeat frames are carried in an `sNodeHbMessage`, which stores the decoded heartbeat type, the source CAN ID, and the raw payload byte. Helper methods such as `decodeNodeHbFrame(...)` and `makeNodeHbFrame(...)` convert between CAN frames and these heartbeat structures.

The CAN communication service uses the shared `sCanExchange` structure to pass data between threads. Normal elevator events continue to use the regular `receivedEvents` queue, while heartbeat messages are routed into a separate `receivedNodeHbMessages` queue. This keeps heartbeat traffic out of the core state-machine path. The COMMS service also maintains health counters such as the COMMS progress/heartbeat counter, received frame count, dropped event count, transmitted frame count, and transmit failure count so the SC can distinguish between node heartbeat failures and internal COMMS thread failures.

The supervisory application class, `cSupervisoryApplication`, runs the heartbeat timing and validation logic. It tracks whether a heartbeat reply window is open, which nodes are expected to reply, which nodes have replied, and which nodes missed the window using bit masks such as `kNodeHbFc1Mask`, `kNodeHbFc2Mask`, `kNodeHbFc3Mask`, `kNodeHbCcMask`, and `kExpectedNodeHbReplyMask`. Methods such as `processNodeHbCycle(...)`, `startNodeHbRequest()`, and the normal `runControlCycle(...)` handle sending a request every 3 seconds, draining the heartbeat queue every 10 ms control cycle, replying to node heartbeat requests, and reporting a fault if a node does not respond within the 1 second verification window.